MODULE 08

Designing and Implementing RESTful APIs with Flask and FastAPI

Resources and HTTP semantics, Flask and FastAPI services, testing, security and real-time WebSockets

```
>>> app = FastAPI(title="Books API")
```



AIN3005 · Advanced Python Programming

Binnur Kurt, PhD · Bahçeşehir University · AI Engineering

Learning Objectives

By the end of this module, you will be able to:

1. Design resource-oriented URIs and choose methods and status codes that follow HTTP semantics
2. Explain statelessness, content negotiation and HATEOAS, and weigh REST's benefits and drawbacks
3. Build a JSON CRUD API in Flask with routes, converters, error handlers and blueprints
4. Test Flask and FastAPI endpoints with pytest, `test_client()` and `TestClient`
5. Implement validated FastAPI endpoints with type hints, Pydantic v2 models and `Depends`
6. Document and secure APIs with OpenAPI, OAuth2 bearer tokens and JWT
7. Add real-time features with Flask-SocketIO and FastAPI WebSockets
8. Compare Flask and FastAPI and justify a choice for a given project

MODULE 08 · RESTFUL APIS

Module Roadmap

01 **REST Principles**
Resources, representations and the uniform interface of HTTP

02 **Flask Fundamentals**
A microframework: routes, requests, responses and blueprints

03 **Building and Testing a Flask API**
A books CRUD service with pytest, OpenAPI docs, caching and JWT

04 **Real-Time Flask with Socket.IO**
Push events to browsers with Flask-SocketIO 5

05 **FastAPI Fundamentals**
Type hints in — validated JSON and OpenAPI docs out

06 **WebSockets with FastAPI**
Real-time, bidirectional messaging on the ASGI stack

01

REST Principles

Resources, representations and the uniform interface of HTTP

- What REST is — and what it is not
- REST in five steps: IDs, links, methods, representations, statelessness
- Status codes, HATEOAS and common URI patterns
- Benefits, drawbacks and alternatives

What Is REST?

Definition

REST (*Representational State Transfer*) is an *architectural style* for networked systems, defined by Roy T. Fielding in his 2000 doctoral dissertation.

Constraint	What it means for an API
Client–server	User interface and data storage evolve independently
Stateless	Every request carries all the information the server needs
Cacheable	Responses state whether and for how long they may be reused
Uniform interface	Resources with URIs, standard methods, self-describing messages, hypermedia links
Layered system	Proxies, gateways and CDNs can sit between client and server
Code on demand	*Optional:* the server may send code (e.g. JavaScript) to the client

REST in Five Steps

1. **Give everything an ID** — Every resource gets a stable URI
2. **Link things together** — Representations contain links to related resources
3. **Use standard methods** — `GET`, `POST`, `PUT`, `PATCH`, `DELETE` with their HTTP meaning
4. **Multiple representations** — JSON, XML, HTML, CSV … chosen per request
5. **Communicate statelessly** — No server-side session between requests

Note

A practical primer after Stefan Tilkov, *A Brief Introduction to REST* (InfoQ, 2007). The next five slides take one step at a time.

Step 1: Give Every Resource an ID

- A **resource** is anything worth naming: a user, a book, an order, a search
- Its **URI** is a stable, global identifier
- Nouns, not verbs — the HTTP method is the verb
- Plural collections with items below them
- Lowercase, hyphens, no file extensions

URIs

```
https://example.com/users
https://example.com/users/johnDoe
https://example.com/users/{login}
https://example.com/users/johnDoe/orders
```

Avoid	Prefer
`GET /getUser?id=7`	`GET /users/7`
`POST /createOrder`	`POST /orders`
`GET /users/7/delete`	`DELETE /users/7`

Step 2: Link Things Together

GET /users (XML, original)

```xml
<users>
  <user ref="/users/johnDoe"/>
  <user ref="/users/janeDoe"/>
  <user ref="/users/jimKirk"/>
</users>
```

GET /users (JSON)

```json
{
  "users": [
    {"login": "johnDoe", "href": "/users/johnDoe"},
    {"login": "janeDoe", "href": "/users/janeDoe"},
    {"login": "jimKirk", "href": "/users/jimKirk"}
  ],
  "next": "/users?page=2"
}
```

Key idea

Clients *follow* links instead of building URIs by hand, so the server can reorganise its URI space — or paginate — without breaking clients.

Step 3: Use Standard HTTP Methods

Method	Purpose	Safe	Idempotent	Body
`GET`	Read a representation (cacheable)	✓	✓	—
`HEAD`	Like `GET`, but headers only	✓	✓	—
`POST`	Create a subordinate resource or process data	✗	✗	✓
`PUT`	Create or replace the resource at a known URI	✗	✓	✓
`PATCH`	Partially update a resource	✗	not guaranteed	✓
`DELETE`	Remove the resource	✗	✓	optional
`OPTIONS`	Ask which methods are allowed (CORS pre-flight)	✓	✓	optional

Key idea

Safe: the client asks for no state change (read-only).

Idempotent: sending the request once or ten times leaves the server in the same state.

An HTTP Exchange, Line by Line

Request

```
POST /api/v1/books HTTP/1.1
Host: api.example.com
Accept: application/json
Content-Type: application/json
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...

{"title": "Dune", "year": 1965}
```

Response

```
HTTP/1.1 201 Created
Location: /api/v1/books/42
Content-Type: application/json
Content-Length: 41

{"id": 42, "title": "Dune", "year": 1965}
```

Part	Request	Response
Start line	method, path, version	version, status code, reason
Headers	`Host`, `Accept`, `Content-Type`, `Authorization`	`Location`, `Content-Type`, `Cache-Control`
Body	the representation to create	the representation created

1 · REST PRINCIPLES

Status Codes You Will Use Every Day

Classes: `1xx` informational · `2xx` success · `3xx` redirection · `4xx` client error · `5xx` server error

Code	Meaning — typical use
`200` OK	Successful `GET`, `PUT`, `PATCH` with a body
`201` Created	`POST` created a resource; send `Location`
`202` Accepted	Queued for later processing
`204` No Content	Success, empty body (often `DELETE`)
`301`/`308` Moved	Resource has a new permanent URI
`304` Not Modified	Cached copy is still valid

Code	Meaning — typical use
`400` Bad Request	Malformed or invalid input
`401` Unauthorized	Missing or invalid credentials
`403` Forbidden	Authenticated, but not allowed
`404` Not Found	No such resource
`405` Method Not Allowed	e.g. `DELETE` on a read-only resource
`409` Conflict	Clashes with current state (duplicate)
`422` Unprocessable	Well-formed but fails validation
`500` Internal Error	A bug on the server — never on purpose

QUICK CHECK

Pick the Status Code

Choose the most appropriate status code for each situation.

#	Situation	Status?
1	`DELETE /books/7` succeeded; nothing to return	
2	`POST /books` with a body that is not valid JSON	
3	`GET /me` on a protected API without any token	
4	Valid token, but the user may not edit this book	
5	`POST /users` with an e-mail that is already registered	
6	`POST /books` validated and stored as `/books/42`	

ANSWER

Pick the Status Code

#	Situation	Answer	Why
1	Delete succeeded, no body	`204`	Success without content
2	Body is not valid JSON	`400`	Request cannot even be parsed
3	No token	`401`	Authentication missing
4	Token valid, action forbidden	`403`	Authenticated but not authorised
5	E-mail already registered	`409`	Conflicts with existing state
6	Created `/books/42`	`201` + `Location`	A new resource exists

Key idea

`401` asks *who are you?*, `403` says *you may not*. For duplicates, `409` and `422` are both defensible — what matters is a `4xx`, because the client must change the request.

1 · REST PRINCIPLES

Step 4: Support Multiple Representations

- A **resource** is abstract; clients exchange **representations** of it
- The client asks with `Accept`; the server labels the reply with `Content-Type`
- Unsupported format → `406 Not Acceptable`; URI suffixes (`/books/42.json`) are a common shortcut

JSON representation

```
GET /books/42 HTTP/1.1
Accept: application/json

HTTP/1.1 200 OK
Content-Type: application/json

{"id": 42, "title": "Dune"}
```

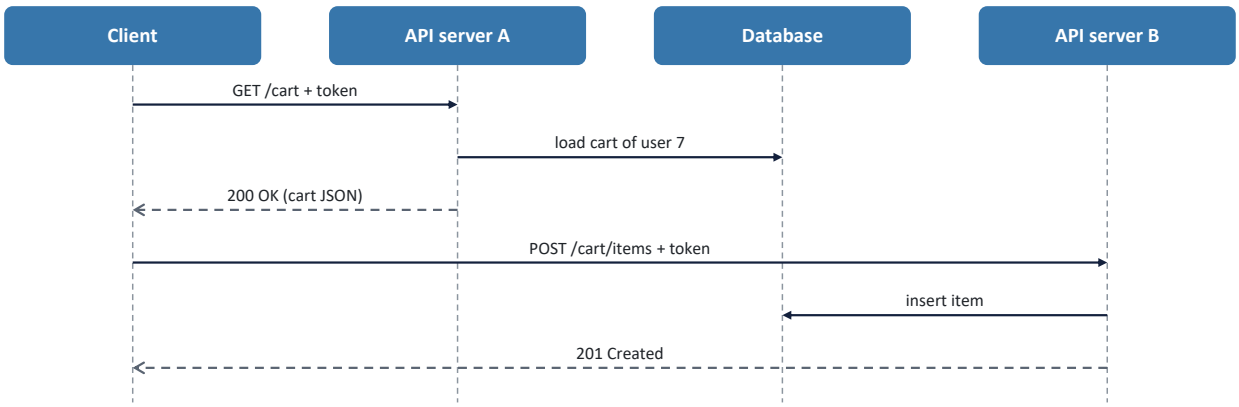
CSV representation

```
GET /books/42 HTTP/1.1
Accept: text/csv

HTTP/1.1 200 OK
Content-Type: text/csv

id,title
42,Dune
```

Step 5: Communicate Statelessly



Key idea

Server A kept nothing between requests, so server B could answer the next one: every request carries its credentials and context; state lives in the client or the database.

HATEOAS: Hypermedia Drives the Client

- *Hypermedia as the Engine of Application State*: one fixed entry point, e.g. `GET /`
- Clients discover resources and allowed actions by following links
- The links present *are* the permitted state transitions

order 17 — pending

```
{
  "id": 17,
  "status": "pending",
  "_links": {
    "self": {"href": "/orders/17"},
    "pay": {"href": "/orders/17/payment"},
    "cancel": {"href": "/orders/17/cancel"}
  }
}
```

order 17 — paid

```
{
  "id": 17,
  "status": "paid",
  "_links": {
    "self": {"href": "/orders/17"},
    "receipt": {"href": "/orders/17/receipt"}
  }
}
```

Note

Most public APIs stop at level 2 of Richardson's maturity model (resources + methods + status codes); HATEOAS is level 3.

Container–Item and Map–Key Patterns

Container–item: the *server* chooses item URIs

Operation	Request	Success
List	`GET /books`	`200`
Add	`POST /books`	`201` + `Location`
Read	`GET /books/42`	`200`
Replace	`PUT /books/42`	`200/204`
Remove	`DELETE /books/42`	`204`

Map–key: the *client* chooses the keys

Operation	Request	Success
List	`GET /files`	`200`
Put new	`PUT /files/{key}`	`201`
Read	`GET /files/{key}`	`200`
Update	`PUT /files/{key}`	`200/204`
Remove	`DELETE /files/{key}`	`204`

Key idea

Who controls the URI space decides the method: `POST` to a container, `PUT` to a key — e.g. `PUT /files/docs/contents.xml`.

TRY IT

Design the URI Scheme

A university library needs an API. For each use case, write **method + URI** and the **success status code**.

1. List all books, optionally only those by one author
2. Show one book and where to find its copies
3. Register a new member
4. Member 12 borrows copy 5 of book 42
5. Member 12 returns that copy
6. List the current loans of member 12

Tip

Hint: model actions as resources — borrowing a copy *creates a loan*.

ANSWER

Design the URI Scheme

#	Endpoint	Success
1	`GET /books?author=Knuth`	`200`
2	`GET /books/42` (links to `/books/42/copies`)	`200`
3	`POST /members`	`201` + `Location`
4	`POST /loans` body `{"member": 12, "copy": "42-5"}`	`201` + `Location`
5	`PATCH /loans/901` body `{"returned": true}`	`200`
6	`GET /members/12/loans?active=true`	`200`

Key idea

Verbs such as *borrow* and *return* become resources (a loan) and state changes on them; filtering uses query parameters, not new URIs.

1 · REST PRINCIPLES

Benefits of the RESTful Approach

Server side

- Horizontal scaling: stateless servers are interchangeable
- Simple failover and load balancing
- Cacheable responses cut load and latency
- Loose coupling between client and server

Client side

- Bookmarkable, linkable resources
- Try it in a browser, `curl` or Postman
- Any language with an HTTP library
- Choice of formats: JSON, XML, CSV

Ecosystem

- Runs on existing web infrastructure: proxies, CDNs, gateways
- Stable, widely known protocol (HTTP)
- Huge tool and framework support
- Evolves without breaking old clients

Drawbacks and Alternatives

Drawback	Mitigation or alternative
Over- and under-fetching; chatty clients need many calls	`fields=` filters, compound endpoints, or **GraphQL**
No built-in contract language (the original's "no WSDL")	**OpenAPI** now fills that role (sections 3 and 5)
Processes and actions are awkward to model as resources	Action resources (`/orders/17/cancel`) or RPC (**gRPC**)
Long `GET` URLs hit server limits (≈ 8 KB)	`POST /searches` for complex queries
Request/response only; no server push	**WebSockets**, Server-Sent Events (sections 4 and 6)
Security is not built into the style	HTTPS, OAuth2/JWT, rate limiting, input validation

> **Note**
> Module 6 compared REST with SOAP and messaging at the architecture level; here the drawbacks are practical design costs.

02

Flask Fundamentals

A microframework: routes, requests, responses and blueprints

- Philosophy, installation and a first app
- Routing, variable rules and HTTP methods
- Reading requests and building responses
- Errors, URL building, static files, blueprints and the app factory

2 · FLASK FUNDAMENTALS

Flask: A Microframework

Lightweight core
Routing, requests, responses, templates — little else to learn

Not opinionated
You choose the database, forms and authentication libraries

Explicit over implicit
Routes, configuration and extensions are visible in your code

Extensible
Extensions add features: Flask-SQLAlchemy, Flask-Login, Flask-SocketIO

No built-in ORM
Use SQLAlchemy, PyMongo (Module 7) or plain SQL

Flexible structure
One file for a prototype; blueprints and a factory for large apps

> **Note**
> Built on **Werkzeug** (WSGI toolkit: requests, responses, routing) and **Jinja** (templates). *Micro* means a small core, not a toy.

2 · FLASK FUNDAMENTALS

Installing Flask and Saying Hello

TERMINAL (WINDOWS: .VENV\SCRIPTS\ACTIVATE)

```
$ python -m venv .venv
$ source .venv/bin/activate
$ pip install flask
$ flask --version
Python 3.12.4
Flask 3.1.3
Werkzeug 3.1.8
```

TERMINAL

```
$ flask --app hello run --debug
 * Running on http://127.0.0.1:5000
$ curl http://127.0.0.1:5000/
Hello, World!
```

hello.py

```python
from flask import Flask

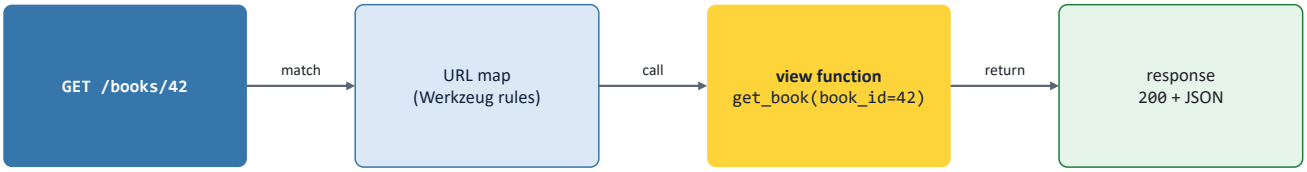
app = Flask(__name__)


@app.route("/")
def hello_world() -> str:
    return "Hello, World!"


if __name__ == "__main__":
    app.run(debug=True)
```

> **Watch out**
> The built-in server is for development only. In production use a WSGI server such as **Gunicorn** or **Waitress**.

Routing: From URL to View Function




```python
from flask import Flask

app = Flask(__name__)


@app.route("/")
@app.route("/home")
def home() -> str:
    return "Welcome to the home page!"


for rule in app.url_map.iter_rules():
    print(f"{rule.rule:24} -> {rule.endpoint}")
```

OUTPUT

```
/static/<path:filename>  -> static
/home                    -> home
/                        -> home
```


Variable Rules and Converters

```python
@app.get("/books/<int:book_id>")
def get_book(book_id: int) -> dict:
    return {"id": book_id,
            "type": type(book_id).__name__}


@app.get("/files/<path:name>")
def get_file(name: str) -> dict[str, str]:
    return {"name": name}


client = app.test_client()
print(client.get("/books/42").get_json())
print(client.get("/books/abc").status_code)
print(client.get("/files/docs/a.md").get_json())
```

OUTPUT

```
{'id': 42, 'type': 'int'}
404
{'name': 'docs/a.md'}
```

Converter	Accepts
`string`	default: any text without `/`
`int`	non-negative integers
`float`	non-negative floats
`path`	like `string`, but slashes allowed
`uuid`	UUID strings

Tip

`app.test_client()` sends requests to the app in-process — no server needed. We use it to show real outputs.

Restricting HTTP Methods

```python
from flask import request


@app.route("/books", methods=["GET", "POST"])
def books() -> tuple[dict, int]:
    if request.method == "POST":
        return {"created": True}, 201
    return {"books": []}, 200


@app.delete("/books/<int:book_id>")  # shortcut since Flask 2.0
def delete_book(book_id: int) -> tuple[str, int]:
    return "", 204


client = app.test_client()
print(client.post("/books").status_code)
print(client.put("/books").status_code)
wrong = client.get("/books/1")
print(wrong.status_code, wrong.headers["Allow"])
```

OUTPUT

```
201
405
405 DELETE, OPTIONS
```


Reading the Request

```python
from flask import request


@app.post("/search")
def search() -> dict:
    data = request.get_json(silent=True) or {}
    page = request.args.get("page", 1, type=int)
    agent = request.headers.get("User-Agent", "")
    return {"query": data.get("q"), "page": page,
            "client": agent.split("/")[0]}


client = app.test_client()
reply = client.post("/search?page=2", json={"q": "flask"})
print(reply.get_json())
print(client.post("/search?page=two").get_json())
```

OUTPUT

```
{'client': 'Werkzeug', 'page': 2, 'query': 'flask'}
{'client': 'Werkzeug', 'page': 1, 'query': None}
```

Attribute	Contains
`request.get_json()`	parsed JSON body
`request.args`	query-string parameters
`request.form`	HTML form fields
`request.headers`	request headers
`request.files`	uploaded files
`request.method`	`"GET"`, `"POST"`, …

Building Responses

View returns	Flask sends
`dict` or `list`	JSON, status `200`
`str`	`text/html`, status `200`
`(body, status)`	body with that status
`(body, status, headers)`	… plus extra headers
`jsonify(...)`	a JSON `Response` object
`make_response(...)`	a `Response` you can modify

```python
@app.get("/ping")
def ping() -> dict[str, str]:
    return {"status": "ok"}


@app.post("/items")
def create_item() -> tuple[dict, int, dict]:
    return {"id": 7}, 201, {"Location": "/items/7"}


client = app.test_client()
for reply in (client.get("/ping"),
              client.post("/items")):
    print(reply.status_code, reply.mimetype,
          reply.get_json())
```

OUTPUT

```
200 application/json {'status': 'ok'}
201 application/json {'id': 7}
```


TRACE THE CODE

What Does the Client Receive?

```python
from flask import Flask, abort

app = Flask(__name__)
STOCK = {"pen": 3, "ink": 0}


@app.get("/stock/<item>")
def stock(item: str) -> dict:
    if item not in STOCK:
        abort(404)
    return {"item": item, "count": STOCK[item]}


client = app.test_client()
print(client.get("/stock/pen").get_json()["count"])
print(client.get("/stock/ink").status_code)
print(client.get("/stock/cap").status_code)
print(client.get("/stock/pen/").status_code)
print(client.post("/stock/pen").status_code)
```

ANSWER

What Does the Client Receive?

OUTPUT

```
3
200
404
404
405
```

- Line 16: `ink` exists — a count of 0 is data, not an error → `200`
- Line 17: `cap` is not in `STOCK` → `abort(404)`
- Line 18: the rule has no trailing slash, so `/stock/pen/` matches nothing → `404`
- Line 19: only `GET` is registered → `405 Method Not Allowed`

2 · FLASK FUNDAMENTALS

Redirects and abort

```python
from flask import Response, abort, redirect, url_for

BOOKS = {1: {"id": 1, "title": "Dune"}}


@app.get("/book/<int:book_id>")  # old URL
def old_book_url(book_id: int) -> Response:
    return redirect(url_for("get_book", book_id=book_id), 301)


@app.get("/books/<int:book_id>")
def get_book(book_id: int) -> dict:
    if book_id not in BOOKS:
        abort(404)  # raises NotFound, the view stops here
    return BOOKS[book_id]


client = app.test_client()
moved = client.get("/book/1")
print(moved.status_code, moved.headers["Location"])
print(client.get("/book/1", follow_redirects=True).get_json())
print(client.get("/books/9").status_code)
```

OUTPUT

```
301 /books/1
{'id': 1, 'title': 'Dune'}
404
```

Consistent JSON Error Responses

`HTTPException` is imported from `werkzeug.exceptions`; the hidden route `/boom` raises `RuntimeError`.

```python
@app.errorhandler(HTTPException)
def http_error(error: HTTPException) -> tuple[dict, int]:
    body = {"error": error.name, "detail": error.description}
    return body, error.code


@app.errorhandler(Exception)
def unexpected_error(error: Exception) -> tuple[dict, int]:
    app.logger.exception("unhandled error")  # traceback to the log
    return {"error": "Internal Server Error"}, 500


client = app.test_client()
missing = client.get("/nowhere")
print(missing.status_code, missing.get_json()["error"])
print(client.get("/boom").get_json())
```

OUTPUT

```
404 Not Found
{'error': 'Internal Server Error'}
```


URL Building and Static Files

```python
from flask import url_for


@app.get("/books/<int:book_id>")
def get_book(book_id: int) -> dict[str, int]:
    return {"id": book_id}


with app.test_request_context():
    print(url_for("get_book", book_id=42))
    print(url_for("get_book", book_id=4, v=2))
    print(url_for("get_book", book_id=4,
                  _external=True))
    print(url_for("static",
                  filename="css/style.css"))
```

OUTPUT

```
/books/42
/books/4?v=2
http://localhost/books/4
/static/css/style.css
```

project layout

```
books_app/
├── app.py
├── static/
│   ├── css/style.css
│   └── img/logo.png
└── templates/
    └── index.html
```

Key idea

Build URLs from **endpoint names**, never by string concatenation: renaming a route then updates every link.

2 · FLASK FUNDAMENTALS

Blueprints: Modular Applications

books/routes.py

```python
from flask import Blueprint

books_bp = Blueprint("books", __name__)


@books_bp.get("/")
def list_books() -> list[str]:
    return ["Dune", "Emma"]


@books_bp.get("/<int:book_id>")
def get_book(book_id: int) -> dict[str, int]:
    return {"id": book_id}
```

app.py

```python
from flask import Flask

from books.routes import books_bp

app = Flask(__name__)
app.register_blueprint(books_bp,
                       url_prefix="/api/v1/books")

client = app.test_client()
print(client.get("/api/v1/books/").get_json())
print(client.get("/api/v1/books/3").get_json())
```

OUTPUT

```
['Dune', 'Emma']
{'id': 3}
```

Key idea

A **blueprint** is a set of routes, error handlers and static files that is registered on an app later — usually one per feature, with its own `url_prefix`.

2 · FLASK FUNDAMENTALS

The Application Factory Pattern

```python
import os

from flask import Flask


class Config:
    SECRET_KEY = os.environ.get("BOOKS_SECRET_KEY", "dev-only")


class TestConfig(Config):
    TESTING = True


def create_app(config: type[Config] = Config) -> Flask:
    """Build a new, fully configured application."""
    app = Flask(__name__)
    app.config.from_object(config)
    app.register_blueprint(books_bp, url_prefix="/api/v1/books")
    return app


app = create_app(TestConfig)
print(app.config["TESTING"], app.config["SECRET_KEY"])
```

OUTPUT

```
True dev-only
```

FIND THE BUG

Find the Bugs in the Countries API

This service (from the original deck, reformatted) runs, but it has at least **four** problems. Find them.

```
1  import json
2
3  from flask import Flask, jsonify
4  from pymongo import MongoClient
5
6  client = MongoClient()
7  client = MongoClient("mongodb://localhost:27017")
8  countries = client["world"]["countries"]
9  app = Flask(__name__)
10
11
12 @app.route("/world/api/v1.0/countries/<code>", methods=["GET"])
13 def countryByCode(code):
14     return jsonify(countries.find_one({"_id": code}))
15
16
17 @app.route("/world/api/v1.0/countries", methods=["GET"])
18 def all_countries():
19     return json.dumps([c for c in countries.find({})])
```


ANSWER

Find the Bugs in the Countries API

- Line 14: unknown code → `find_one` returns `None` → `200 null`; must be `404`
- Line 19: `json.dumps` returns a `str` → sent as `text/html`
- Line 19: the whole collection in one response — paginate
- Lines 6–7: two clients (two connection pools); read the URI from config
- Line 13: `countryByCode` → `snake_case`; `methods=["GET"]` is the default

```
@app.get("/world/api/v1/countries/<code>")
def country_by_code(code: str) -> dict:
    country = countries.find_one(
        {"_id": code.upper()})
    if country is None:
        abort(404, description=f"no {code}")
    return country


@app.get("/world/api/v1/countries")
def list_countries() -> list[dict]:
    limit = request.args.get("limit", 50,
                             type=int)
    cursor = countries.find().limit(limit)
    return list(cursor)
```

03

Building and Testing a Flask API

A books CRUD service with pytest, OpenAPI docs, caching and JWT

- Design the endpoints, then implement create, read, update and delete
- Test with `app.test_client()` and pytest fixtures
- Document with OpenAPI (flask-restx, APIFlask, flask-smorest)
- Best practices, caching with Redis and JWT authentication

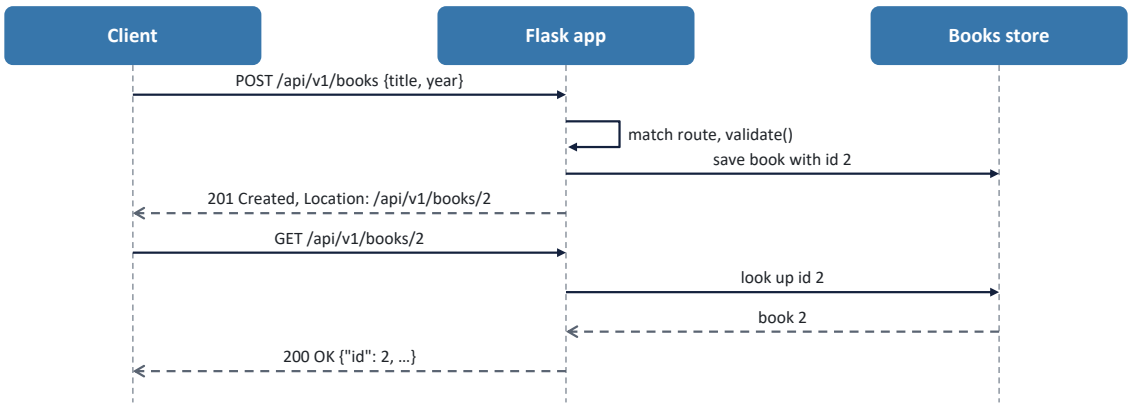
Books API: Resource Design

Endpoint	Action	Success	Errors
`GET /api/v1/books`	List all books	`200`	—
`POST /api/v1/books`	Create a book (server assigns `id`)	`201` + `Location`	`400`
`GET /api/v1/books/{id}`	Read one book	`200`	`404`
`PUT /api/v1/books/{id}`	Replace a book	`200`	`400`, `404`
`DELETE /api/v1/books/{id}`	Delete a book	`204`	`404`
`PUT/DELETE /api/v1/books`	Replace or delete the whole collection	not offered	`405`

Fixed from the original

The original example let `POST /book/{isbn}` create a book and `PUT /book` replace the whole list. Legal HTTP, but surprising: `POST` targets the collection, `PUT` targets an item.

Request Flow: Client, API and Store



Key idea

Today the store is a `dict`; replacing it with MongoDB (Module 7) changes only the store layer, not the routes or the tests.

Books API: Listing and Reading

books_api.py (1/3)

```python
from itertools import count

from flask import Flask, abort, request, url_for

app = Flask(__name__)
BOOKS = {1: {"id": 1, "title": "Fluent Python", "year": 2022}}
_ids = count(start=2)  # next free id


@app.get("/api/v1/books")
def list_books() -> list[dict]:
    return list(BOOKS.values())


@app.get("/api/v1/books/<int:book_id>")
def get_book(book_id: int) -> dict:
    if book_id not in BOOKS:
        abort(404, description=f"book {book_id} not found")
    return BOOKS[book_id]
```

- `BOOKS` is an in-memory store: a restart loses data
- `count(start=2)` hands out unique ids
- `<int:book_id>` converts and rejects non-integers
- `abort(404, description=...)` stops the view; the error handler (part 3) turns it into JSON

Books API: Creating a Book

books_api.py (2/3)

```python
def validate(data: object) -> dict:
    """Return a clean book dict or abort with 400 Bad Request."""
    if not isinstance(data, dict):
        abort(400, description="body must be a JSON object")
    title, year = data.get("title"), data.get("year")
    if not isinstance(title, str) or not title.strip():
        abort(400, description="'title' must be a non-empty string")
    if not isinstance(year, int) or isinstance(year, bool):
        abort(400, description="'year' must be an integer")
    return {"title": title.strip(), "year": year}


@app.post("/api/v1/books")
def create_book() -> tuple[dict, int, dict[str, str]]:
    book = {"id": next(_ids), **validate(request.get_json(silent=True))}
    BOOKS[book["id"]] = book
    location = url_for("get_book", book_id=book["id"])
    return book, 201, {"Location": location}
```

Key idea

`201 Created` + a `Location` header built with `url_for` — clients never guess the new URI.

Books API: Replacing and Deleting

books_api.py (3/3)

```python
from werkzeug.exceptions import HTTPException


@app.put("/api/v1/books/<int:book_id>")
def replace_book(book_id: int) -> dict:
    if book_id not in BOOKS:
        abort(404, description=f"book {book_id} not found")
    data = validate(request.get_json(silent=True))
    BOOKS[book_id] = {"id": book_id, **data}
    return BOOKS[book_id]


@app.delete("/api/v1/books/<int:book_id>")
def delete_book(book_id: int) -> tuple[str, int]:
    if BOOKS.pop(book_id, None) is None:
        abort(404, description=f"book {book_id} not found")
    return "", 204


@app.errorhandler(HTTPException)
def json_error(error: HTTPException) -> tuple[dict, int]:
    return {"error": error.name, "detail": error.description}, error.code
```

Driving the API with the Test Client

```
client = app.test_client()
created = client.post("/api/v1/books",
                      json={"title": "Python Tricks", "year": 2017})
print(created.status_code, created.headers["Location"])
print(client.get("/api/v1/books/2").get_json())
print(client.post("/api/v1/books", json={"year": 2017}).get_json())
update = {"title": "Python Tricks, 2nd ed.", "year": 2024}
print(client.put("/api/v1/books/2", json=update).status_code)
print(client.delete("/api/v1/books/2").status_code)
print(client.get("/api/v1/books/2").get_json())
```

OUTPUT

```
201 /api/v1/books/2
{'id': 2, 'title': 'Python Tricks', 'year': 2017}
{'detail': "'title' must be a non-empty string", 'error': 'Bad Request'}
200
204
{'detail': 'book 2 not found', 'error': 'Not Found'}
```


pytest Fixtures for the Books API

conftest.py

```python
import pytest

from books_api import BOOKS, app


@pytest.fixture
def client():
    """Test client; the store is reset after each test."""
    app.config.update(TESTING=True)
    saved = {key: dict(book) for key, book in BOOKS.items()}
    with app.test_client() as test_client:
        yield test_client
    BOOKS.clear()
    BOOKS.update(saved)
```

- `TESTING=True`: exceptions propagate to the test
- Code before `yield` = setup, after = teardown (Module 3)
- Each test starts from the same seeded store
- Larger apps: `create_app(TestConfig)` in the fixture builds a fresh app per test

pytest Test Cases for the Books API

test_books_api.py

```python
import pytest


def test_create_returns_201_and_location(client):
    response = client.post("/api/v1/books",
                           json={"title": "Dune", "year": 1965})
    assert response.status_code == 201
    book_id = response.get_json()["id"]
    assert response.headers["Location"] == f"/api/v1/books/{book_id}"


def test_unknown_book_is_404(client):
    response = client.get("/api/v1/books/999")
    assert response.status_code == 404
    assert response.get_json()["error"] == "Not Found"


@pytest.mark.parametrize("body", [{}, {"title": ""}, {"title": "Dune"}])
def test_invalid_body_is_400(client, body):
    assert client.post("/api/v1/books", json=body).status_code == 400
```

TERMINAL (SAMPLE OUTPUT)

```
$ pytest -q
.....                                [100%]
5 passed in 0.09s
```

Tip

`parametrize` turns one test function into three test cases — one per invalid body.

FIND THE BUG

Find the Bugs in This Create Endpoint

A teammate wrote this version of `create_book`. The tests for single requests pass. Find **three** problems.

```python
@app.post("/api/v1/books")
def create_book() -> tuple[dict, int]:
    book = {"id": len(BOOKS) + 1,
            **validate(request.get_json(silent=True))}
    BOOKS[book["id"]] = book
    return book, 200
```

Tip

Hint: create books A, B and C, delete book 1, then create D.

ANSWER

Find the Bugs in This Create Endpoint

- Line 3: `len(BOOKS) + 1` reuses ids — after a delete, D gets id 3 and **overwrites** C
- Line 6: creation must answer `201 Created`, not `200`
- Line 6: no `Location` header — clients cannot find the new book

```python
@app.post("/api/v1/books")
def create_book() -> tuple[dict, int, dict[str, str]]:
    book = {"id": next(_ids),
            **validate(request.get_json(silent=True))}
    BOOKS[book["id"]] = book
    location = url_for("get_book", book_id=book["id"])
    return book, 201, {"Location": location}
```


3 · BUILDING AND TESTING A FLASK API

Documenting APIs with OpenAPI

- **OpenAPI** (formerly the Swagger Specification) describes paths, parameters, schemas and security in YAML or JSON
- **Swagger UI** and **ReDoc** render it as interactive documentation
- Generate client SDKs, mocks and contract tests from it

openapi.yaml (excerpt)

```yaml
openapi: 3.1.0
info: {title: Books API, version: 1.0.0}
paths:
  /api/v1/books/{book_id}:
    get:
      summary: Read one book
      responses:
        "200": {description: The book}
        "404": {description: Not found}
```

Flask option	Status and style
Flask-RESTPlus	**Dead** (unmaintained since 2019) — do not use
flask-restx	Community fork of RESTPlus: `Resource` classes, models, Swagger UI
APIFlask	Flask + schemas → validation and OpenAPI, decorator style
flask-smorest	marshmallow schemas, blueprints, OpenAPI 3

Note

FastAPI (section 5) generates OpenAPI and Swagger UI without any extension.

flask-restx: Resources, Models and Swagger UI

tasks_api.py (setup)

```python
from itertools import count

from flask import Flask
from flask_restx import Api, Resource, fields

app = Flask(__name__)
api = Api(app, title="Tasks API", doc="/docs")
ns = api.namespace("tasks")
task = api.model("Task", {
    "id": fields.Integer(readonly=True),
    "title": fields.String(required=True),
})
TASKS: list[dict] = []
_ids = count(start=1)
```

tasks_api.py (resource)

```python
@ns.route("/")
class TaskList(Resource):
    @ns.marshal_list_with(task)
    def get(self) -> list[dict]:
        """List all tasks."""
        return TASKS

    @ns.expect(task, validate=True)
    @ns.marshal_with(task, code=201)
    def post(self) -> tuple[dict, int]:
        """Create a task."""
        new = {"id": next(_ids), **api.payload}
        TASKS.append(new)
        return new, 201
```

Tip

`pip install flask-restx` — Swagger UI is served at `/docs`, generated from the models and docstrings.

REST API Best Practices

Version the API

`/api/v1/...` (or a header); never break v1 clients — add v2

Paginate collections

`?limit=20&offset=40` or cursors; return `next` links and a total

Validate at the boundary

Reject bad input early with `400`/`422` and a clear message

One error format

e.g. RFC 9457 *problem details*: `type`, `title`, `status`, `detail`

Nouns, methods, status codes

`POST /orders`, not `/createOrder`; `201`, `204`, `404` …

Thin views, grouped by feature

A blueprint per feature; business logic in services

Tip

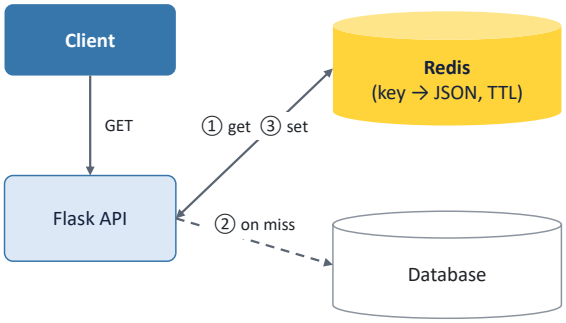
Configuration and secrets come from the environment (application factory, section 2).

Caching Strategies with Redis

```python
from flask import Flask
from flask_caching import Cache

app = Flask(__name__)
cache = Cache(app, config={
    "CACHE_TYPE": "RedisCache",
    "CACHE_REDIS_URL": "redis://localhost:6379/0",
    "CACHE_DEFAULT_TIMEOUT": 60,
})


@app.get("/api/v1/countries")
@cache.cached(timeout=300)
def list_countries() -> list[dict]:
    return load_countries()  # slow query
```



Watch out

Invalidate on writes (`cache.delete(...)`) or keep TTLs short.

Securing Endpoints with JWT

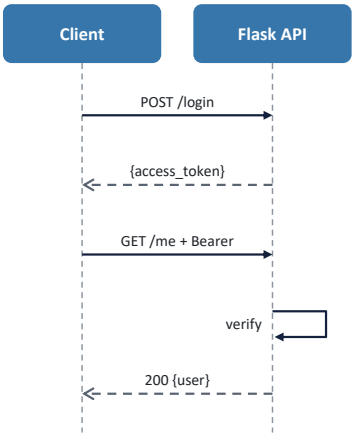
```python
import os

from flask import Flask, request
from flask_jwt_extended import (JWTManager, create_access_token,
                                get_jwt_identity, jwt_required)

app = Flask(__name__)
app.config["JWT_SECRET_KEY"] = os.environ["JWT_SECRET_KEY"]
jwt = JWTManager(app)


@app.post("/api/v1/login")
def login() -> tuple[dict, int]:
    data = request.get_json(silent=True) or {}
    if not check_password(data.get("username"), data.get("password")):
        return {"error": "Unauthorized"}, 401
    token = create_access_token(identity=data["username"])
    return {"access_token": token}, 200


@app.get("/api/v1/me")
@jwt_required()
def me() -> dict[str, str]:
    return {"user": get_jwt_identity()}
```



QUIZ

Quick Check: JWT Signatures

A server issues a signed JWT (algorithm HS256). What does the signature guarantee?

- **A** The payload is encrypted and unreadable without the secret key
- **B** The payload has not been modified since the server signed it
- **C** The token cannot be stolen or replayed by an attacker
- **D** The user is still logged in on the server

ANSWER

Quick Check: JWT Signatures

A server issues a signed JWT (algorithm HS256). What does the signature guarantee?

- **A** The payload is encrypted and unreadable without the secret key
- **B** The payload has not been modified since the server signed it
- **C** The token cannot be stolen or replayed by an attacker
- **D** The user is still logged in on the server

★ **Why**

A signed JWT is *encoded* (Base64URL), not encrypted: anyone can read the claims. The signature only proves integrity and origin. Use HTTPS, short expiry times and no secrets in the payload.

04

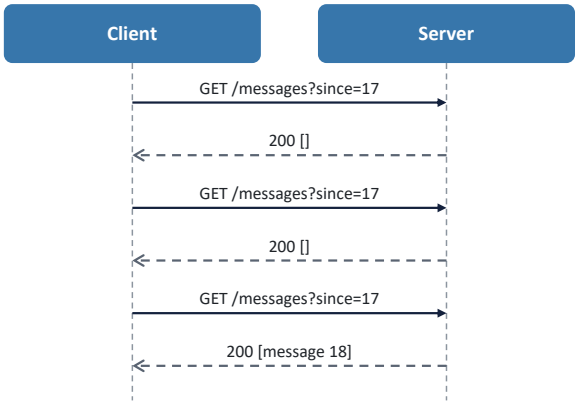
Real-Time Flask with Socket.IO

Push events to browsers with Flask-SocketIO 5

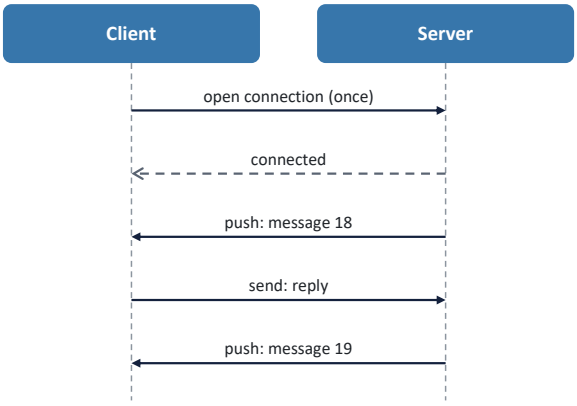
- Why request/response is not enough for real-time features
- Socket.IO events, broadcasting and a chat application
- Namespaces, rooms, testing and deployment

Why Real Time? Polling vs Push

HTTP polling — ask again and again



Persistent connection — the server pushes



Use cases: chat, live notifications, dashboards, collaborative editing, multiplayer games

Socket.IO: Events on Top of WebSocket

A protocol, not just WebSocket

Own framing on top of WebSocket — a plain WebSocket client cannot talk to a Socket.IO server

Transport fallback

Starts with HTTP long-polling, upgrades to WebSocket when possible

Named events

`emit("chat", data)` and `on("chat", handler)` with JSON payloads

Reliability

Automatic reconnection, heartbeats and acknowledgements

Grouping

Namespaces split an application; **rooms** group clients

Two sides

JavaScript client in the browser; Flask-SocketIO (python-socketio) on the server

Setting Up Flask-SocketIO

app.py

```python
import os

from flask import Flask, render_template
from flask_socketio import SocketIO

app = Flask(__name__)
app.config["SECRET_KEY"] = os.getenv("SECRET_KEY", "dev")
socketio = SocketIO(app)


@app.route("/")
def index() -> str:
    return render_template("index.html")


if __name__ == "__main__":
    socketio.run(app, debug=True)
```

TERMINAL

```
$ pip install flask-socketio \
      simple-websocket
```

★ **Key idea**

`socketio.run(app)` replaces `app.run()`: it starts a server that understands WebSocket upgrades.

i **Note**

Flask-SocketIO 5.x pairs with the Socket.IO JavaScript client 3.x/4.x.

Connecting and Exchanging Events

server (Python)

```python
from flask import request
from flask_socketio import emit


@socketio.on("connect")
def on_connect() -> None:
    print("connected:", request.sid)


@socketio.on("message")
def handle_message(text: str) -> None:
    print(f"Received message: {text}")
    emit("reply", f"Server got: {text}")
```

client (templates/index.html)

```html
<script src="/static/socket.io.min.js"></script>
<script>
  const socket = io();  // same origin
  socket.on("connect", () => {
    console.log("Connected:", socket.id);
    socket.emit("message", "Hello, Server!");
  });
  socket.on("reply", (text) => {
    console.log(text);
  });
</script>
```

Key idea

`emit` inside a handler replies to the **sender** only; each client has a session id `request.sid`.

Broadcasting Messages

server

```python
@socketio.on("broadcast")
def handle_broadcast(text: str) -> None:
    emit("broadcast", text, broadcast=True)


@app.post("/api/v1/news")
def post_news() -> tuple[dict, int]:
    news = request.get_json(silent=True) or {}
    socketio.emit("news", news)  # all clients
    return news, 201
```

client

```javascript
socket.on("broadcast", (text) => {
  console.log(`Broadcast: ${text}`);
});
```

Call	Recipients
`emit(ev, data)`	the sender only
`emit(ev, data, broadcast=True)`	every client in the namespace
`emit(..., broadcast=True, include_self=False)`	everyone except the sender
`emit(ev, data, to=room)`	clients in that room
`socketio.emit(ev, data)`	all clients — works anywhere, e.g. in a REST view

Chat App: Server

chat.py

```python
from flask import Flask, render_template
from flask_socketio import SocketIO, emit

app = Flask(__name__)
socketio = SocketIO(app)


@app.route("/")
def index() -> str:
    return render_template("chat.html")


@socketio.on("send_message")
def handle_send_message(data: dict) -> None:
    text = str(data.get("text", "")).strip()[:500]
    if text:
        message = {"user": data.get("user", "anonymous"), "text": text}
        emit("receive_message", message, broadcast=True)


if __name__ == "__main__":
    socketio.run(app)
```


Chat App: Browser Client

templates/chat.html

```html
<ul id="messages"></ul>
<input id="text"> <button id="send">Send</button>
<script src="/static/socket.io.min.js"></script>
<script>
  const socket = io();
  socket.on("receive_message", (msg) => {
    const item = document.createElement("li");
    item.textContent = `${msg.user}: ${msg.text}`;
    document.getElementById("messages").append(item);
  });
  document.getElementById("send").onclick = () => {
    const input = document.getElementById("text");
    socket.emit("send_message", {user: "ada", text: input.value});
    input.value = "";
  };
</script>
```

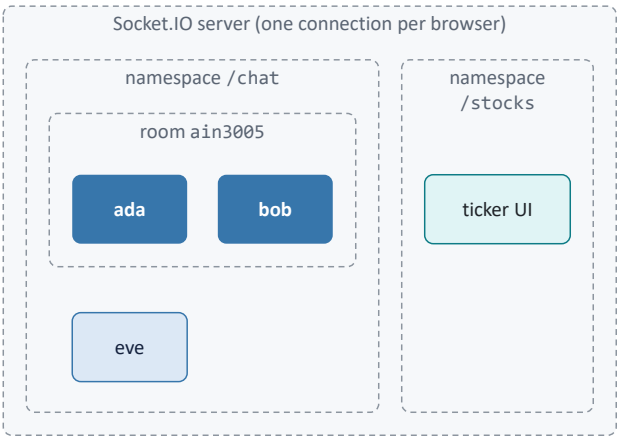
Watch out

Use `textContent`, never `innerHTML`, for chat text — otherwise a message like `<img src=x onerror=...>` runs as script in every browser (XSS).

Tip

Event names must match the server exactly: `send_message` out, `receive_message` in.

Namespaces and Rooms



server

```python
@socketio.on("message", namespace="/chat")
def chat_message(text: str) -> None:
    emit("message", text, broadcast=True)


@socketio.on("subscribe", namespace="/stocks")
def subscribe(symbol: str) -> None:
    emit("price", {"symbol": symbol,
                   "price": 101.5})
```

client

```javascript
const chat = io("/chat");
const stocks = io("/stocks");
```


Rooms: Targeted Groups

```python
from flask_socketio import emit, join_room, leave_room


@socketio.on("join")
def on_join(data: dict) -> None:
    join_room(data["room"])
    emit("status", f"{data['user']} joined", to=data["room"])


@socketio.on("leave")
def on_leave(data: dict) -> None:
    leave_room(data["room"])
    emit("status", f"{data['user']} left", to=data["room"])


@socketio.on("send_to_room")
def handle_room_event(data: dict) -> None:
    emit("room_message", data["text"], to=data["room"])
```

Note

`to=` replaces the older `room=` argument. Every client is also alone in a private room named after its `request.sid`.

Testing Socket.IO Handlers

test_chat.py

```python
from chat import app, socketio


def test_message_is_broadcast():
    ada = socketio.test_client(app)
    bob = socketio.test_client(app)
    assert ada.is_connected()

    ada.emit("send_message",
             {"user": "ada", "text": "hi"})

    received = bob.get_received()
    assert received[0]["name"] == "receive_message"
    assert received[0]["args"][0] == {
        "user": "ada", "text": "hi"}
```

- `socketio.test_client(app)` connects a fake client — no server, no browser
- `emit` sends an event as that client
- `get_received()` returns the events queued for a client
- Same pytest workflow as for REST endpoints (Module 3)

Deploying Flask-SocketIO

TERMINAL

```
$ pip install gunicorn simple-websocket
$ gunicorn -w 1 --threads 100 chat:app  # threading mode
$ pip install gevent
$ gunicorn -k gevent -w 1 chat:app      # gevent mode
```

Update

The original `eventlet` worker class still works, but eventlet is deprecated — prefer threading or gevent.

One worker per process

Socket.IO state lives in memory: `-w 1`, or sticky sessions for several workers

Scale out with a queue

`SocketIO(app, message_queue="redis://")` shares events between processes and hosts

Proxy WebSockets

nginx must forward the `Upgrade` and `Connection` headers

Lock it down

`cors_allowed_origins=[...]`, HTTPS/WSS, authenticate on `connect`

FIND THE BUG

Why Does Only the Sender See the Message?

Two browser tabs run the chat page. Tab A sends a message, but only tab A displays it. Find the bug, fix it, then change the fix so the **sender does not** receive its own copy.

```python
@socketio.on("send_message")
def handle_send_message(data: dict) -> None:
    emit("receive_message", data)
```



ANSWER

Why Does Only the Sender See the Message?

- Inside a handler, `emit` without options replies to the **sender only**
- Fix: `broadcast=True` → every client in the namespace receives it
- `include_self=False` excludes the sender (it shows its own message locally)

```python
@socketio.on("send_message")
def handle_send_message(data: dict) -> None:
    emit("receive_message", data, broadcast=True)


@socketio.on("send_message_to_others")
def handle_send_to_others(data: dict) -> None:
    emit("receive_message", data, broadcast=True,
         include_self=False)
```

05

FastAPI Fundamentals

Type hints in — validated JSON and OpenAPI docs out

- Path, query and body parameters from type hints
- Pydantic v2 models, response models, status codes and errors
- Dependency injection, async endpoints, routers and middleware
- Authentication, docs, testing, lifespan and performance

Why FastAPI?

Type hints drive everything

Parameters, bodies and responses are converted and validated from annotations

Pydantic v2 models

Declarative schemas with a fast, Rust-powered validation core

Automatic docs

OpenAPI schema plus Swagger UI and ReDoc out of the box

Dependency injection

`Depends` shares auth, database sessions and pagination

Async native (ASGI)

`async def` endpoints on Starlette + Uvicorn; plain `def` works too

Editor support

Autocompletion and type checking catch errors before run time

Version note

Current FastAPI requires **Python 3.10+** (the original slide said 3.6+); this course uses 3.12+ and FastAPI 0.115 or newer.

Installing FastAPI and a First App

main.py

```python
from fastapi import FastAPI

app = FastAPI()


@app.get("/")
def read_root() -> dict[str, str]:
    return {"message": "Hello, World"}
```

TERMINAL

```
$ pip install "fastapi[standard]"
$ fastapi dev main.py
# or: uvicorn main:app --reload
INFO: Uvicorn running on http://127.0.0.1:8000
$ curl http://127.0.0.1:8000/
{"message":"Hello, World"}
```



1. Import `FastAPI`
2. Create the `app` instance
3. Declare routes with `@app.get(...)`
4. Run with an ASGI server (Uvicorn)

Path Parameters with Type Hints

```python
from fastapi import FastAPI
from fastapi.testclient import TestClient

app = FastAPI()


@app.get("/items/{item_id}")
def read_item(item_id: int) -> dict[str, int]:
    return {"item_id": item_id}


client = TestClient(app)
print(client.get("/items/42").json())
bad = client.get("/items/abc")
print(bad.status_code, bad.json()["detail"][0]["type"])
```

OUTPUT

```
{'item_id': 42}
422 int_parsing
```

- `{item_id}` in the path + `item_id: int` → value converted to `int`
- Invalid input → automatic `422` with a JSON explanation
- The same hint documents the parameter in OpenAPI
- `TestClient` (built on HTTPX) calls the app in-process

Query Parameters and Validation

```python
from typing import Annotated

from fastapi import Query


@app.get("/books")
def list_books(
    skip: int = 0,
    limit: Annotated[int, Query(ge=1, le=50)] = 10,
    q: str | None = None,
) -> dict:
    return {"skip": skip, "limit": limit, "q": q}


print(client.get("/books").json())
print(client.get("/books?skip=20&q=python").json())
print(client.get("/books?limit=500").status_code)
```

OUTPUT

```
{'skip': 0, 'limit': 10, 'q': None}
{'skip': 20, 'limit': 10, 'q': 'python'}
422
```

- Parameters not in the path are **query parameters**
- A default value makes them optional
- `str | None = None`: may be absent
- `Annotated[int, Query(ge=1, le=50)]` adds constraints and docs

> **Watch out**
> The original `q: str = None` lies to the type checker; write `str | None`.

Request Bodies with Pydantic Models

```python
from pydantic import BaseModel, Field


class BookIn(BaseModel):
    title: str = Field(min_length=1, max_length=200)
    year: int = Field(ge=1450, le=2100)
    tags: list[str] = Field(default_factory=list)


@app.post("/books", status_code=201)
def create_book(book: BookIn) -> BookIn:
    return book


ok = client.post("/books", json={"title": "Dune", "year": 1965})
print(ok.status_code, ok.json())
bad = client.post("/books", json={"title": "", "year": 1965})
print(bad.status_code, bad.json()["detail"][0]["loc"])
```

OUTPUT

```
201 {'title': 'Dune', 'year': 1965, 'tags': []}
422 ['body', 'title']
```

5 · FASTAPI FUNDAMENTALS

Pydantic v2 Essentials

```python
from pydantic import BaseModel, Field, ValidationError


class Book(BaseModel):
    title: str
    year: int = Field(ge=1450)


book = Book.model_validate({"title": "Dune", "year": "1965"})
print(repr(book.year))
print(book.model_dump())
print(book.model_dump_json())
try:
    Book(title="Scroll", year=100)
except ValidationError as error:
    print(error.errors()[0]["type"])
```

OUTPUT

```
1965
{'title': 'Dune', 'year': 1965}
{"title":"Dune","year":1965}
greater_than_equal
```

Pydantic v1	Pydantic v2
`.dict()`	`.model_dump()`
`.json()`	`.model_dump_json()`
`Book.parse_obj(d)`	`Book.model_validate(d)`
`.copy(update=...)`	`.model_copy(update=...)`
`class Config:`	`model_config = ConfigDict(...)`
`@validator`	`@field_validator`

5 · FASTAPI FUNDAMENTALS

Response Models: Control the Output

models

```python
from pydantic import BaseModel


class UserIn(BaseModel):
    username: str
    password: str


class UserOut(BaseModel):
    username: str
    is_active: bool = True
```

endpoint

```python
@app.post("/users", response_model=UserOut,
          status_code=201)
def create_user(user: UserIn) -> dict:
    record = user.model_dump()
    record["password_hash"] = "hashed"  # demo
    return record  # contains secrets ...


body = {"username": "ada", "password": "s3cret"}
print(client.post("/users", json=body).json())
```

OUTPUT

```
{'username': 'ada', 'is_active': True}
```

Key idea

`response_model` validates **and filters** the return value: fields not in `UserOut` never leave the server.

5 · FASTAPI FUNDAMENTALS

Status Codes and HTTPException

```python
from fastapi import HTTPException, status

BOOKS = {1: {"id": 1, "title": "Dune"}}


@app.get("/books/{book_id}")
def read_book(book_id: int) -> dict:
    if book_id not in BOOKS:
        raise HTTPException(status.HTTP_404_NOT_FOUND,
                            detail=f"Book {book_id} not found")
    return BOOKS[book_id]


@app.delete("/books/{book_id}", status_code=204)
def delete_book(book_id: int) -> None:
    if BOOKS.pop(book_id, None) is None:
        raise HTTPException(404, detail="Book not found")


print(client.get("/books/7").json())
print(client.delete("/books/1").status_code)
print(client.delete("/books/1").status_code)
```

OUTPUT

```
{'detail': 'Book 7 not found'}
204
404
```


TRACE THE CODE

Predict the Status Codes

```python
class Order(BaseModel):
    item: str
    quantity: int = Field(gt=0)


@app.post("/orders", status_code=201)
def create_order(order: Order) -> Order:
    return order


pen = {"item": "pen", "quantity": 2}
print(client.post("/orders", json=pen).status_code)
print(client.post("/orders", json=pen | {"quantity": 0}).status_code)
print(client.post("/orders", json=pen | {"quantity": "3"}).json())
print(client.get("/orders").status_code)
print(client.post("/orders/7", json=pen).status_code)
```

ANSWER

Predict the Status Codes

OUTPUT

```
201
422
{'item': 'pen', 'quantity': 3}
405
404
```

- Line 12: valid → the declared `201`; line 13: `quantity` must be `> 0` → `422`
- Line 14: lax mode converts `"3"` to `3` — accepted
- Line 15: the path exists but has no `GET` → `405`; line 16: no such path → `404`

5 · FASTAPI FUNDAMENTALS

Dependency Injection with Depends

shared dependency

```python
from typing import Annotated

from fastapi import Depends, Query


def pagination(
    skip: int = 0,
    limit: Annotated[int, Query(ge=1, le=100)] = 10,
) -> dict[str, int]:
    return {"skip": skip, "limit": limit}


Page = Annotated[dict[str, int], Depends(pagination)]
```

two endpoints

```python
@app.get("/books")
def list_books(page: Page) -> dict:
    return {"resource": "books", **page}


@app.get("/authors")
def list_authors(page: Page) -> dict:
    return {"resource": "authors", **page}


print(client.get("/books?limit=5").json())
print(client.get("/authors?skip=10").json())
```

OUTPUT

```
{'resource': 'books', 'skip': 0, 'limit': 5}
{'resource': 'authors', 'skip': 10, 'limit': 10}
```

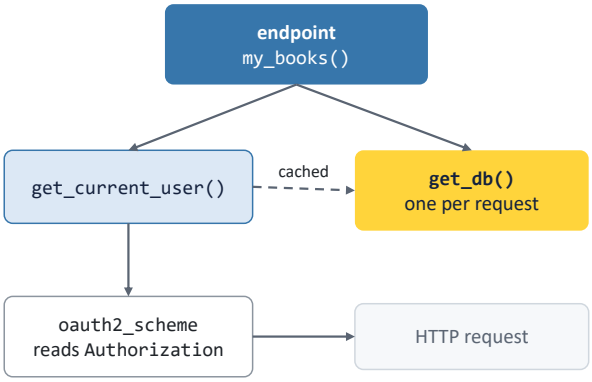
★ A dependency is any callable: FastAPI calls it per request and injects the result.

Dependencies Form a Graph

```python
def get_db() -> Iterator[Session]:
    ...  # yield a database session


def get_current_user(
    token: Annotated[str, Depends(oauth2_scheme)],
    db: Annotated[Session, Depends(get_db)],
) -> User: ...


@app.get("/me/books")
def my_books(
    user: Annotated[User, Depends(get_current_user)],
    db: Annotated[Session, Depends(get_db)],
) -> list[Book]: ...
```



Key idea

FastAPI resolves the graph for each request and calls each dependency **once**, even if several nodes need it (`use_cache=True`).

Dependencies with yield and Overrides

```python
def get_db() -> Iterator[dict]:
    print("open session")
    try:
        yield {"books": ["Dune"]}
    finally:
        print("close session")


@app.get("/books")
def list_books(db: Annotated[dict, Depends(get_db)]) -> list[str]:
    return db["books"]


print(client.get("/books").json())
app.dependency_overrides[get_db] = lambda: {"books": ["Test Book"]}
print(client.get("/books").json())
```

OUTPUT

```
open session
close session
['Dune']
['Test Book']
```

Note

A `yield` dependency is a context manager in disguise (Module 4): code after `yield` runs when the request is finished.

5 · FASTAPI FUNDAMENTALS

Async Endpoints and the Event Loop

```python
import asyncio
import time


async def query(value: object) -> object:
    await asyncio.sleep(0.1)  # stands in for real I/O
    return value


@app.get("/dashboard")
async def dashboard() -> dict:
    start = time.perf_counter()
    user, orders = await asyncio.gather(query("ada"),
                                        query(3))
    took = round(time.perf_counter() - start, 1)
    return {"user": user, "orders": orders,
            "seconds": took}


print(client.get("/dashboard").json())
```

OUTPUT

```
{'user': 'ada', 'orders': 3, 'seconds': 0.1}
```

Endpoint	Runs …
`async def`	on the event loop — `await` non-blocking I/O
`def`	in a worker thread pool — blocking calls are OK

Watch out

Never call blocking functions (`time.sleep`, `requests.get`, a sync DB driver) inside `async def`: they freeze **every** request.

QUIZ

Quick Check: Blocking the Event Loop

Which endpoint blocks FastAPI's event loop for one second, delaying all other requests?

A `def handler(): time.sleep(1)`

B `async def handler(): await asyncio.sleep(1)`

C `async def handler(): time.sleep(1)`

D `async def handler(): await http.get(url)` with an async HTTP client

ANSWER

Quick Check: Blocking the Event Loop

Which endpoint blocks FastAPI's event loop for one second, delaying all other requests?

A `def handler(): time.sleep(1)`

B `async def handler(): await asyncio.sleep(1)`

C `async def handler(): time.sleep(1)`

D `async def handler(): await http.get(url)` with an async HTTP client

Why

FastAPI runs `def` endpoints in a thread pool (A only occupies one worker thread). In C the coroutine calls a blocking function directly on the event loop, so nothing else can run for one second. B and D yield control while they wait.

5 · FASTAPI FUNDAMENTALS

Structuring an App with APIRouter

routers/books.py

```python
from fastapi import APIRouter

router = APIRouter(prefix="/books", tags=["books"])


@router.get("/")
def list_books() -> list[str]:
    return ["Dune", "Emma"]


@router.get("/{book_id}")
def read_book(book_id: int) -> dict[str, int]:
    return {"book_id": book_id}
```

main.py

```python
from fastapi import FastAPI
from fastapi.testclient import TestClient

from routers import books

app = FastAPI()
app.include_router(books.router, prefix="/api/v1")

client = TestClient(app)
print(client.get("/api/v1/books/7").json())
print([route.path for route in app.routes
       if route.path.startswith("/api")])
```

OUTPUT

```
{'book_id': 7}
['/api/v1/books/', '/api/v1/books/{book_id}']
```

Key idea

`APIRouter` is FastAPI's blueprint: routes, dependencies and tags per feature, assembled with `include_router`.

5 · FASTAPI FUNDAMENTALS

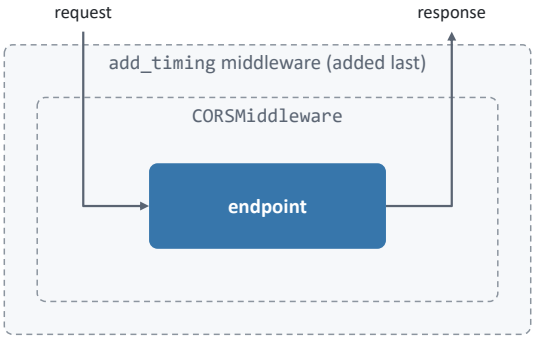
Middleware and CORS

```python
import time

from fastapi import Request
from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=["https://app.example.com"],
    allow_methods=["GET", "POST"],
    allow_headers=["Authorization", "Content-Type"],
)


@app.middleware("http")
async def add_timing(request: Request, call_next):
    start = time.perf_counter()
    response = await call_next(request)
    elapsed = time.perf_counter() - start
    response.headers["X-Process-Time"] = f"{elapsed:.4f}"
    return response
```



Watch out

The original `allow_origins=["*"]` with `allow_credentials=True` lets **any** website call the API with users' cookies — list trusted origins.

5 · FASTAPI FUNDAMENTALS

Serving Static Files

```python
from pathlib import Path

from fastapi import FastAPI
from fastapi.staticfiles import StaticFiles
from fastapi.testclient import TestClient

Path("static").mkdir(exist_ok=True)
Path("static/style.css").write_text("body { margin: 0; }\n",
                                    encoding="utf-8")

app = FastAPI()
app.mount("/static", StaticFiles(directory="static"), name="static")

client = TestClient(app)
response = client.get("/static/style.css")
print(response.status_code, response.headers["content-type"])
print(response.text.strip())
print(app.url_path_for("static", path="style.css"))
```

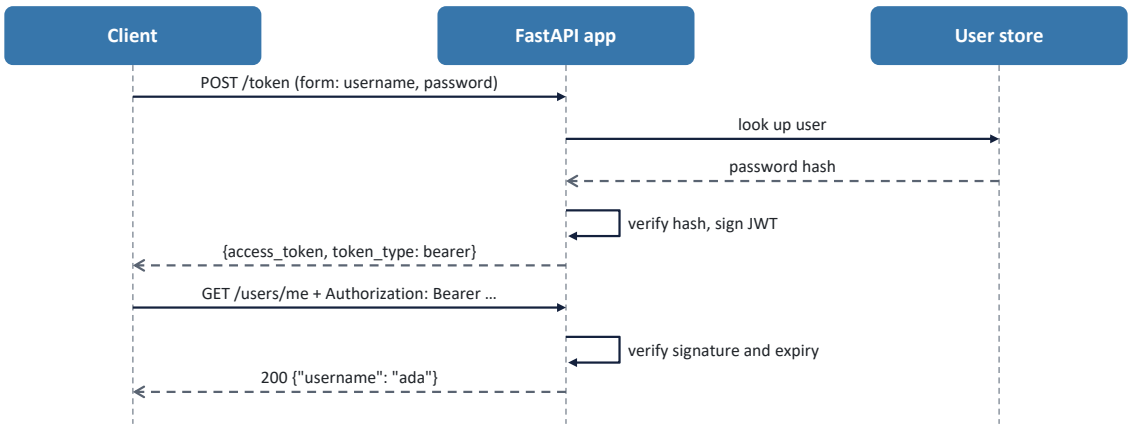
OUTPUT

```
200 text/css; charset=utf-8
body { margin: 0; }
/static/style.css
```

Note

`mount` attaches a whole sub-application at a path prefix; in production a web server or CDN usually serves static files.

OAuth2 Password Flow



`OAuth2PasswordBearer` extracts the bearer token; Swagger UI then shows an **Authorize** button.

Protecting Endpoints with a Bearer Token

```python
from fastapi.security import OAuth2PasswordBearer

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")


def current_user(token: Annotated[str, Depends(oauth2_scheme)]) -> str:
    if not token.startswith("demo-"):  # real apps: verify a JWT
        raise HTTPException(401, detail="Invalid token",
                            headers={"WWW-Authenticate": "Bearer"})
    return token.removeprefix("demo-")


@app.get("/users/me")
def read_me(user: Annotated[str, Depends(current_user)]) -> dict:
    return {"username": user}


print(client.get("/users/me").json())
auth = {"Authorization": "Bearer demo-ada"}
print(client.get("/users/me", headers=auth).json())
```

OUTPUT

```
{'detail': 'Not authenticated'}
{'username': 'ada'}
```

Issuing and Verifying JWTs

```python
import os
from datetime import UTC, datetime, timedelta

import jwt  # pip install pyjwt

SECRET_KEY = os.environ["JWT_SECRET_KEY"]  # never in the code


def create_access_token(username: str) -> str:
    expires = datetime.now(UTC) + timedelta(minutes=30)
    claims = {"sub": username, "exp": expires}
    return jwt.encode(claims, SECRET_KEY, algorithm="HS256")


def current_user(token: Annotated[str, Depends(oauth2_scheme)]) -> str:
    try:
        claims = jwt.decode(token, SECRET_KEY, algorithms=["HS256"])
    except jwt.InvalidTokenError:  # bad signature or expired
        raise HTTPException(401, detail="Invalid token") from None
    return claims["sub"]
```

✓ POST `/token` verifies the password hash (`pwdlib`), then returns `create_access_token(name)` as `access_token`.

Interactive Docs: Swagger UI and ReDoc

```python
from fastapi import FastAPI

app = FastAPI(title="Books API", version="1.0.0")


@app.get("/books/{book_id}", tags=["books"],
         summary="Read one book")
def read_book(book_id: int) -> dict[str, int]:
    return {"book_id": book_id}


schema = app.openapi()
print(schema["openapi"], schema["info"]["title"])
print(list(schema["paths"]))
```

OUTPUT

```
3.1.0 Books API
['/books/{book_id}']
```

URL	Shows
`/docs`	**Swagger UI**: try requests in the browser
`/redoc`	**ReDoc**: readable reference
`/openapi.json`	the raw OpenAPI 3.1 schema

✓ **Tip**

Tags, summaries, docstrings and models all flow into the docs. Hide them in production with `FastAPI(docs_url=None, redoc_url=None)` if needed.

5 · FASTAPI FUNDAMENTALS

Testing FastAPI with TestClient

test_main.py

```python
from fastapi.testclient import TestClient

from main import app, get_db


client = TestClient(app)


def test_read_root():
    assert client.get("/").json() == {"message": "Hello, World"}


def test_unknown_book_is_404():
    assert client.get("/books/99").status_code == 404


def test_list_books_with_fake_db():
    app.dependency_overrides[get_db] = lambda: {"books": ["Test"]}
    try:
        response = client.get("/books")
    finally:
        app.dependency_overrides.clear()
    assert response.json() == ["Test"]
```

TERMINAL (SAMPLE OUTPUT)

```
$ pytest -q
...                                 [100%]
3 passed in 0.31s
```

Tip

Override dependencies instead of patching: fake database, fixed user, no network.

5 · FASTAPI FUNDAMENTALS

Lifespan: Startup and Shutdown

```python
from collections.abc import AsyncIterator
from contextlib import asynccontextmanager

from fastapi import FastAPI, Request


@asynccontextmanager
async def lifespan(app: FastAPI) -> AsyncIterator[None]:
    print("startup: open connection pool")
    app.state.pool = {"size": 5}  # e.g. AsyncMongoClient(...)
    yield
    print("shutdown: close connection pool")


app = FastAPI(lifespan=lifespan)


@app.get("/pool")
def pool_info(request: Request) -> dict:
    return request.app.state.pool


with TestClient(app) as client:
    print(client.get("/pool").json())
```

OUTPUT

```
startup: open connection pool
{'size': 5}
shutdown: close connection pool
```

Update

`@app.on_event("startup")` and `"shutdown"` are deprecated — use `lifespan`.

5 · FASTAPI FUNDAMENTALS

Performance Notes

ASGI stack
Uvicorn (server) → Starlette (routing, WebSockets) → FastAPI (validation, DI)

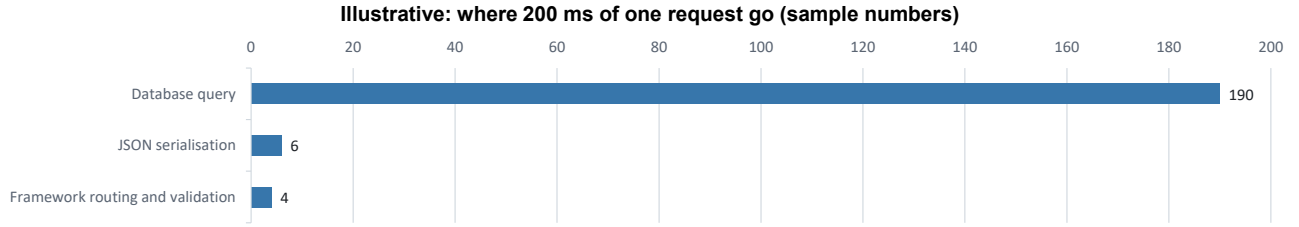
Pydantic v2 core
Validation and serialisation run in compiled Rust code (`pydantic-core`)

async helps I/O
Thousands of concurrent waits per process; CPU-bound work still needs processes (Module 5)

Scale with workers
`fastapi run --workers 4` or several Uvicorn processes behind a proxy

Watch the hot path
Blocking calls in `async def`, N+1 database queries, huge responses

Measure, do not guess
Load-test (Locust, `wrk`) and profile (Module 11) before optimising



TRY IT

Add a PATCH Endpoint

Books are stored as `Book` models (`title: str`, `year: int`). Add `PATCH /books/{book_id}` that updates **only the fields the client sends** and returns the updated book (404 if unknown).

```
>>> client.patch("/books/1", json={"year": 1966}).json()
{'title': 'Dune', 'year': 1966}
```

Tip

Hints: a second model with optional fields; `model_dump(exclude_unset=True)`; `model_copy(update=...)`.

ANSWER

Add a PATCH Endpoint

```python
class Book(BaseModel):
    title: str
    year: int


class BookPatch(BaseModel):
    title: str | None = None
    year: int | None = None


BOOKS = {1: Book(title="Dune", year=1965)}


@app.patch("/books/{book_id}")
def patch_book(book_id: int, patch: BookPatch) -> Book:
    if book_id not in BOOKS:
        raise HTTPException(404, detail="Book not found")
    changes = patch.model_dump(exclude_unset=True)
    BOOKS[book_id] = BOOKS[book_id].model_copy(update=changes)
    return BOOKS[book_id]


print(client.patch("/books/1", json={"year": 1966}).json())
```

OUTPUT

```
{'title': 'Dune', 'year': 1966}
```

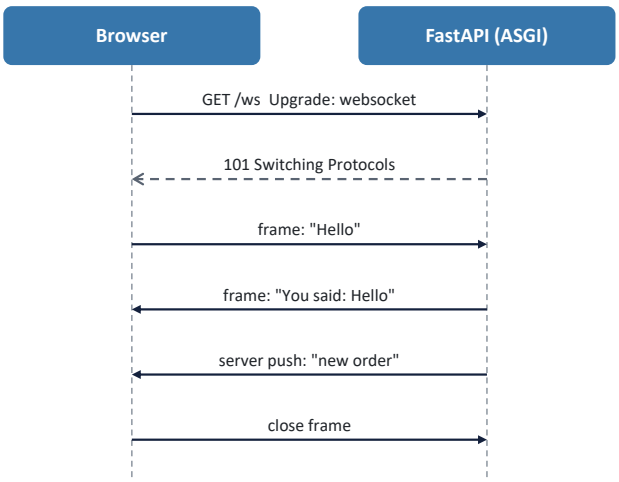

06

WebSockets with FastAPI

Real-time, bidirectional messaging on the ASGI stack

- The WebSocket handshake and when to use it
- A basic endpoint and a JavaScript client
- Managing many connections and broadcasting
- Testing WebSockets with `TestClient`

The WebSocket Handshake



- Starts as HTTP, then **upgrades** (`101`) to a persistent TCP connection
- **Full duplex**: both sides send whenever they want
- Small frames instead of full HTTP requests — low latency
- `ws://` or `wss://` (TLS) URLs
- For chat, live prices, games, notifications

A Basic WebSocket Endpoint

```python
from fastapi import FastAPI, WebSocket
from fastapi.testclient import TestClient

app = FastAPI()


@app.websocket("/ws")
async def websocket_endpoint(websocket: WebSocket) -> None:
    await websocket.accept()
    await websocket.send_text("Welcome to WebSocket!")
    data = await websocket.receive_text()
    await websocket.send_text(f"You said: {data}")
    await websocket.close()


client = TestClient(app)
with client.websocket_connect("/ws") as ws:
    print(ws.receive_text())
    ws.send_text("Hello, Server!")
    print(ws.receive_text())
```

OUTPUT

```
Welcome to WebSocket!
You said: Hello, Server!
```

Connecting from JavaScript and wscat

browser

```
const socket = new WebSocket("ws://localhost:8000/ws");

socket.onopen = () => {
  console.log("Connected");
  socket.send("Hello, Server!");
};
socket.onmessage = (event) => {
  console.log("Message:", event.data);
};
socket.onclose = () => console.log("Disconnected");
```

TERMINAL

```
$ npx wscat -c ws://localhost:8000/ws
Connected (press CTRL+C to quit)
< Welcome to WebSocket!
> Hello, Server!
< You said: Hello, Server!
```

Tip

Browser DevTools → Network → **WS** → Messages shows every frame.

Watch out

Fixed from the original: `send()` right after `new WebSocket(...)` fails — the connection is not open yet. Send inside `onopen`.

A Connection Manager for Many Clients

manager.py

```
from fastapi import WebSocket


class ConnectionManager:
    """Keeps track of the open WebSocket connections."""

    def __init__(self) -> None:
        self.active: list[WebSocket] = []

    async def connect(self, websocket: WebSocket) -> None:
        await websocket.accept()
        self.active.append(websocket)

    def disconnect(self, websocket: WebSocket) -> None:
        self.active.remove(websocket)

    async def broadcast(self, message: str) -> None:
        for connection in list(self.active):
            await connection.send_text(message)
```

- One object remembers every open `WebSocket`
- `accept()` first, then register the connection
- `broadcast` iterates over a **copy** of the list
- `disconnect` must be called when a client leaves

Note

The original appended the socket *before* `accept()` and never broadcast; the manager makes both explicit.

Broadcasting to Every Client

main.py

```python
from fastapi import FastAPI, WebSocket, WebSocketDisconnect

app = FastAPI()
manager = ConnectionManager()


@app.websocket("/chat/{name}")
async def chat(websocket: WebSocket, name: str) -> None:
    await manager.connect(websocket)
    await manager.broadcast(f"{name} joined")
    try:
        while True:
            text = await websocket.receive_text()
            await manager.broadcast(f"{name}: {text}")
    except WebSocketDisconnect:
        manager.disconnect(websocket)
        await manager.broadcast(f"{name} left")
```

! Several worker processes = several managers: fan out via a broker (Redis pub/sub, Module 9).

Testing WebSockets with TestClient

```python
from fastapi.testclient import TestClient

with TestClient(app) as client:
    with client.websocket_connect("/chat/ada") as ada:
        print("ada:", ada.receive_text())
        with client.websocket_connect("/chat/bob") as bob:
            print("ada:", ada.receive_text())
            print("bob:", bob.receive_text())
            bob.send_text("hi all")
            print("ada:", ada.receive_text())
            print("bob:", bob.receive_text())
        print("ada:", ada.receive_text())
print("open connections:", len(manager.active))
```

OUTPUT

```
ada: ada joined
ada: bob joined
bob: bob joined
ada: bob: hi all
bob: bob: hi all
ada: bob left
open connections: 0
```

★ **Key idea**

Nested `with` blocks = overlapping connections; leaving a block closes that connection and triggers `WebSocketDisconnect` on the server.

FIND THE BUG

Find the Bug in the Chat Loop

The chat works while two users are connected. After one of them closes the tab, the other user's next message makes *their* connection fail too. Why? Fix it.

```
1  clients: list[WebSocket] = []
2
3
4  @app.websocket("/ws")
5  async def chat(websocket: WebSocket) -> None:
6      await websocket.accept()
7      clients.append(websocket)
8      while True:
9          text = await websocket.receive_text()
10         for client in clients:
11             await client.send_text(text)
```


ANSWER

Find the Bug in the Chat Loop

- When a user leaves, line 9 raises `WebSocketDisconnect` in *their* handler — nothing catches it
- Their closed socket stays in `clients`
- The next broadcast (line 11) sends to the closed socket and raises in the *sender's* handler

```
@app.websocket("/ws")
async def chat(websocket: WebSocket) -> None:
    await websocket.accept()
    clients.append(websocket)
    try:
        while True:
            text = await websocket.receive_text()
            for client in list(clients):
                await client.send_text(text)
    except WebSocketDisconnect:
        clients.remove(websocket)
```

MODULE 08 · COMPARISON

Flask vs FastAPI

Aspect	Flask 3.x	FastAPI
Interface	WSGI; sync first (async views possible)	ASGI; async first (sync in a thread pool)
Validation	By hand or extension (marshmallow, APIFlask)	Built in: type hints + Pydantic v2
API docs	Extension: flask-restx, APIFlask, flask-smorest	Built in: OpenAPI, Swagger UI, ReDoc
Shared logic	Request context, `g`, extensions, decorators	`Depends` dependency graph
Real time	Flask-SocketIO (Socket.IO protocol)	Native WebSockets (Starlette)
Testing	`app.test_client()` + pytest	`TestClient` + pytest, dependency overrides
Best fit	Server-rendered sites + APIs, rich plugin ecosystem	JSON APIs and microservices, high I/O concurrency

Key idea

The REST design — resources, URIs, methods, status codes — is identical in both; only the framework idioms differ.

MODULE 08 · PRACTICE

Practice Problems

1. Library API in Flask

Members, books and loans with blueprints, JSON errors and pytest tests for every status code

2. Port it to FastAPI

Pydantic models, response models and a `Depends`-based pagination

3. Secure it

JWT login; only the borrower may return a loan — test `401` versus `403`

4. Persist it

Replace the dict with MongoDB: `AsyncMongoClient` created in `lifespan` (Module 7)

5. Make it live

Broadcast "copies available" over WebSockets whenever a loan changes

6. Cache it

`ETag` + `If-None-Match` on `GET /books/{id}`, answering `304 Not Modified`

MODULE 08 · SUMMARY

Key Takeaways

- ✓ REST models a system as resources with URIs, manipulated with standard methods and status codes
- ✓ Safe and idempotent methods, statelessness and hypermedia make APIs cacheable, scalable and evolvable
- ✓ Flask gives explicit building blocks: routes, converters, `request`, error handlers, blueprints
- ✓ FastAPI derives validation, serialisation and OpenAPI docs from type hints and Pydantic v2 models
- ✓ `Depends`, async endpoints and `lifespan` manage shared resources; pytest with test clients verifies every contract
- ✓ Socket.IO and WebSockets add real-time push where request/response is not enough

NEXT UP Module 09 — Event-Driven Architecture